

UC Davis Ride Sharing

STA141B Final Project Report

Winter 2022

Group 27: Jianing Li, Walid El Mouahidi, Min Seo Park

1 Introduction

The objective of this project is to analyze the posts of the Facebook group “UC Davis Ride Sharing” to understand and predict the users’ behaviors. “UC Davis Ride Sharing” is a private Facebook group created for the purpose of organizing shared trips for users with common destinations. By analyzing the posts on this group, some of the main information to acquire are “most preferred/searched start and end location for drivers and passengers”, “most preferred dates/periods/times that drivers and passengers organize/search for rides”, “price ranges offered by drivers”, and “most common dates/periods/times that users post on the Facebook group about these trips”.

2 Data Set

The “UC Davis Ride Sharing” group has a specific rule when it comes to posting. If a user is willing to drive others from location A to B on a certain day (e.g. 02/13/2022), then this user makes a post with a hashtag, #drive02132022. Contrary to the driver, the passenger, or the user that needs a ride from location A to B on a certain day (e.g. 02/13/2022), posts with a hashtag, #ride02132022. For all users, they also include in their posts the start location, destination, time of the ride. Additionally, most drivers also include the price they charge to each of the passengers on their posts.

Driver	Rider
--------	-------

The final data set after data transformation will be a data frame with each post as a row, each with information in following columns:

- *content*: original post message
- *rideordrive*: integer indicator 0 or 1; 0 if the post is from a rider, 1 if the post is from a driver
- *dateofride*: date of the ride as datetime object
- *price*: integer indeicating price offered by the user
- *location*: string value represents the starting location; returns NaN is not found by the algorithm
- *destination*: string value represents ending location (destination); returns NaN is not found by the algorithm
- *dayofpost*: date of the post as datetime object; returns NaN is not found by the algorithm
- *timeofpost*: time of the post as datetime object; returns NaN is not found by the algorithm
- *weekdayofpost*: string value represents the days of the week; returns NaN is not found by the algorithm

Each post scraped from this group is formatted slightly differently. Most posts follow to rules with #drive or #ride, but other crucial information such as the start location and destination are formatted as the user desires. Due to this variation in posts, it is challenging to design a language processing that can transform the raw scraped data into the final data set with 100% accuracy.

3 Web Scrapping

Web scraping private Facebook group is a challenging process, as Facebook doesn’t provide API and changes the HTML class codes periodically. The scraping algorithm consisted in three main blocks:

- opening the Facebook webpage by using the Selenium package and input the login information: username and password; then click on the login button and accept the cookies.
- open the UC Davis Ride Sharing Facebook group and on intervals of 1 second scroll down and click on each post to expand it, as Facebook pages are dynamic pages that upload more content each time the user scrolls down and as the long posts are reduced in size so they can fit the page design/structure requirements.
- scrape each post content and creation date by using BeautifulSoup package.

```
In [1]: import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import scipy
import time as tm
import requests
import lxml
import bs4 as BeautifulSoup
import lxml.html as lx
%pylab inline
import selenium
from selenium import webdriver
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.support.wait import WebDriverWait
from selenium.webdriver.support.ui import Select
from webdriver_manager.chrome import ChromeDriverManager

#importing time libraries to add wait times
from datetime import datetime
from time import sleep

#importing beautiful soup to read the page html source code
from bs4 import BeautifulSoup
```

Populating the interactive namespace from numpy and matplotlib

```
In [2]: #Open facebook and Login

driver = webdriver.Chrome(r"C:\Users\user\Desktop\chromedriver.exe")
driver.get("https://www.facebook.com")

#target username
username = WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.CSS_SELECTOR,
password = WebDriverWait(driver, 10).until(EC.element_to_be_clickable((By.CSS_SELECTOR,

#enter username and password
username.clear()
username.send_keys("username")
password.clear()
password.send_keys("passcode")

#target the login button and click it
button = WebDriverWait(driver, 2).until(EC.element_to_be_clickable((By.CSS_SELECTOR, "b

#We are logged in!
```

```
In [3]: #access the Facebook group
driver.get("https://www.facebook.com/groups/243044995791686")
time.sleep(15)

#scroll the page and expand all the posts

i=0
element=driver.find_element_by_tag_name("body")
Nposts=300
while i<Nposts:
    time.sleep(1)
    element.send_keys(Keys.ARROW_DOWN + "j"+ Keys.ENTER)
    i += 1
```

```
time.sleep(10)
```

```
In [5]: #scrape the posts' content and get the date of creation
soup=BeautifulSoup(driver.page_source,"html.parser")
all_posts=soup.find_all("div",{ "class":"ecm0bbzt hv4rvrfc dati1w0a e5nlhep0"})
links1=soup.find_all("a",{ "class":"oajr1xb2 g5ia77u1 qu0x051f esr5mh6w e9989ue4 r7d6kgc
list_posts=list()
time_posts=list()
for post in all_posts:
    L=post.find("span",{ "class":"d2edcug0 hpfvmrgz qv66sw1b c1et5uql lr9zc1uh a8c37x1j
    list_posts.append(L)
for t in links1:
    T=t["aria-label"]
    time_posts.append(T)
```

```
In [35]: time_post=time_posts[1:len(list_posts)+1]
print(time_post[:10])
```

['15 March 2020', '32 m', '34 m', '38 m', '39 m', '1 d', '48 m', '9 h', '57 m', '1 h']

```
In [14]: list_posts[:10]
```

```
Out[14]: ["Not sure if anyone is interested, but I'm driving tuesday 3/17 at 8:30PM to UCLA. I'm
willing to stop anywhere along the 5 or 405. $35 per seat. small bags only because I do
n't have a lot of trunk space. Message me if interested. #drive03172020",
'#drive03242022Driving Davis -> LA (K-town)Thursday (03/24) at 7:00am Pick Up: Davis (U
mall) Drop Off: Korea Town $55 per person/ 3 spots available **$5 dollars deposit requir
ed to reserve a seatPlease wear mask + Fully Vaccinated only',
'Hi!! Looking for a ride from Davis to Riverside this Thursday 03/17 at 5:00 PM. Please
let me know! #ride03172022',
'#ride03182022 or anytime this week or weekend to Santa Barbra!!! Willing to pitch for
gas ofc',
'Looking for a ride from Davis to Santa Barbara on 3/17, but am willing to be dropped o
ff in SLO, Six Flags/Magic Mountain, or Ventura if that works better for the driver #rid
e03172022',
'#drive03182022Driving Davis->Cupertino/Santa Clara @2:30pmPick up: UmallDrop off: Main
Street Cupertino or Santa Clara Central Park$15 per person (Venmo)2 seats open',
'#drive03182022 Davis->East LA Leaving @5:30PM #drive03202022 East LA -> Davis Leaving
@1 PMFully vaccinated peeps only please. $45 per person. Lmk!',
'Hi! I'm looking for a ride from Davis to UCLA or anywhere in that area or in LA on eit
her Friday or Saturday! (vaccinated + have booster shot + will wear mask) #ride03182022
#ride03192022',
'Driving Davis -> San JoseFriday (3/18) at 3:30pmPick up: UmallDrop off: Safeway at Val
ley Fair $15Message me please keep mask on thru the entire ride',
'#drive03192022Driving Davis —> San Jose (Safeway/Eastridge)Saturday, March 19 @ aroun
d 7/8 pmPickup: Safeway - South Davis (anywhere +$3)Dropoff: Eastridge/Safeway• $15 + Pi
ckup/Dropoff anywhere ($5-$20)• car: 2018 Honda Clarity (new, clean + space for luggage)
~ pls message for a spot !!• must be fully vaccinated + boosted and keep mask on at all
times!• venmo/zelle • 4 seats available']
```

4 Language Processing

In order to obtain the final data set in the desired format, language processing was required for each of the variables. ‘Posts’ column was obtained by the data scraping process and renaming the column to ‘posts’.

```
In [60]: #create dataframe for scraped content

df=pd.DataFrame()
df["index"] = list(range(len(list_posts)))
df["content"]=list_posts
df["timeofposting"]=time_post

df.head(10)
```

```
Out[60]:
```

	index	content	timeofposting
0	0	Not sure if anyone is interested, but I'm driv...	15 March 2020
1	1	#drive03242022Driving Davis -> LA (K-town)Thur...	32 m

	index	content	timeofposting
2	2	Hi!! Looking for a ride from Davis to Riversid...	34 m
3	3	#ride03182022 or anytime this week or weekend ...	38 m
4	4	Looking for a ride from Davis to Santa Barbara...	39 m
5	5	#drive03182022Driving Davis->Cupertino/Santa C...	1 d
6	6	#drive03182022 Davis->East LA Leaving @5:30PM...	48 m
7	7	Hi! I'm looking for a ride from Davis to UCLA ...	9 h
8	8	Driving Davis -> San JoseFriday (3/18) at 3:30...	57 m
9	9	#drive03192022Driving Davis ——> San Jose (Safe...	1 h

In [37]:

df.to_csv(r"C:\Users\user\Desktop\data_fin.csv")

4.1 Ride or Drive

Using this ‘posts’ data, the ‘R/D’ data was collected. First, every word in the post is enumerated in reverse order and an empty space is added in front of each hashtags in a post. Then the post is splitted by empty space, which causes all ‘#’s to become the first character of a word. Then for each splitted word, the word is appended to a list if the first character is a ‘#’. After this process, the list includes all hashtags (#drive or #ride). The list is saved as a variable “L” since this list is also required for the ‘date’ column; but for ‘R/D’, use re.sub function to only keep alphabetical characters and a hashtag. When the list goes through this process, it becomes a string with only words in the string being “#drive” or “#ride”. Therefore, using the if-statement to check whether the first five characters of the string is “#driv” or “#ride” can determine whether the post is written by a driver or a passenger, and the indicator is given accordingly. As mentioned, 1 indicates a passenger and 0 indicates a driver.

4.2 Date of Ride

The ‘date’ column shows the date of the ride. This data is obtained from the hashtag (e.g. #drive02132022). However, some users, mostly riders, provide several hashtags to express that the ride can take place on any of these given days. In such cases, the date given in the first hashtag is used for the ‘date’ variable. This may cause the actual trip to be slightly different from the date obtained, but it will be insignificant to the data analysis of this project, since most users provide multiple dates that are one or two days away from each other. In other words, a passenger could provide three options, today, tomorrow, or day after tomorrow, but not tomorrow, next month, or six months from now.

For ‘date’, the same list saved from the ‘R/D’ data is used. Instead of using re.sub to filter only alphabet and hashtag, only numeric values are used for this variable. After this, a few if-statements are required to obtain only the numbers that represent the dates. To do so, all empty strings have to be eliminated, then any string with length less than six, since six numbers are not enough to represent month, date, and year. If string has length six, the numbers could have the format ‘MMDDYY’, even though this format does not follow the rule of this Facebook group. Therefore, expecting that this six-digit string is a date, add “20” in between the fourth and fifth character (e.g. changes 021322 into 02132022). If the string has length eight or more, then only the first eight digits are used. After the string goes through these processes, the eight digit string is converted into a datetime object, with format %m%d%Y. The last process to take is to remove eight digit values that were not meant to be dates. To do this, only the datetime objects with date.year greater than 2012 and less than 2025 were entered into the ‘date’ column. Not all rows have the ‘date’ column filled, since posts that lack hashtags or are in the wrong format cannot be recognized by these codes. Also, there could be slim cases where a post includes a hashtag but the date is written as six digits, and one other numeric value is written later in the post which would cause the string to become length 7, and therefore cannot be recognized by the code as well.

In [61]:

```
#OBJECTIVE: get the ride/drive and date

df2 = pd.DataFrame(index=range(len(df)), columns=range(len(df.columns)))
df2.columns = df.columns

df2['index'] = df['index']
df2['content'] = df['content']
df2['timeofposting'] = df['timeofposting']
df2['rideordrive'] = '' # indicates whether the post is from a driver or rider (driver
df2['dateofride'] = '' # if rider posts more than one hashtag, assume the first hashtag

def hashtag(word): # finds all occurrence of # in string and add empty space in front of
    index = [i for i, w in reversed(list(enumerate(word))) if w == '#'] # List of index

    for i in index:
        word = word[:i] + " " + word[i:]

    return word

for i in range(len(df)):
    l = list()

    w = df.iloc[i, 1]

    w = hashtag(w)
    w = re.split(r" ", w)

    for j in w:
        if len(j)>0 and j[0]=='#':
            l.append(j)
        else:
            None

    alphahash = r'^A-Za-z#+'
    numeric = r'^0-9+' # if trying to take dates from all hashtags in a single post,
    rd = re.sub(alphahash, '', str(l))
    date = re.sub(numeric, '', str(l))

    # code to get 'rideordrive'
    if rd[0:5] == "#driv":
        df2['rideordrive'][i] = 0
    elif rd[0:5] == "#ride":
        df2['rideordrive'][i] = 1
    else:
        None

    # code to get all dates from a single post (not needed for now)
    #date = re.split('#', date)
    #date = ["#" + date for date in date] # add # in front of every element in list
    #date = [ele for ele in date if ele != '#'] # remove every element with only #

    # code to get 'dateofride'
    if len(date) > 0:
        if len(date) > 5: # greater than six since "mmddyy" has length six
            if len(date) == 6: # if element has format 'mmddyy'
                date = date[:4] + "20" + date[4:] # e.g. change year 22 to 2022
            if len(date) > 8: # if element has format other than mmddyyyy
                date = date[:8] # set element to format #mmddyyyy

            date = datetime.strptime(date, '%m%d%Y').date()
            if date.year > 2012 and date.year < 2025: # filter out invalid dates (e.g.
                df2['dateofride'][i] = date

        else:
            date = ''

df2.head(10)
```

C:\Users\user\AppData\Local\Temp\ipykernel_2740\1796024882.py:43: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df2['rideordrive'][i] = 0
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_2740\1796024882.py:63: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
    df2['dateofride'][i] = date
C:\Users\user\AppData\Local\Temp\ipykernel_2740\1796024882.py:45: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
    df2['rideordrive'][i] = 1
```

Out[61]:

	index	content	timeofposting	rideordrive	dateofride
0	0	Not sure if anyone is interested, but I'm driv...	15 March 2020	0	2020-03-17
1	1	#drive03242022Driving Davis -> LA (K-town)Thur...	32 m	0	2022-03-24
2	2	Hi!! Looking for a ride from Davis to Riversid...	34 m	1	2022-03-17
3	3	#ride03182022 or anytime this week or weekend ...	38 m	1	2022-03-18
4	4	Looking for a ride from Davis to Santa Barbara...	39 m	1	2022-03-17
5	5	#drive03182022Driving Davis->Cupertino/Santa C...	1 d	0	2022-03-18
6	6	#drive03182022 Davis->East LA Leaving @5:30PM...	48 m	0	2022-03-18
7	7	Hi! I'm looking for a ride from Davis to UCLA ...	9 h	1	2022-03-18
8	8	Driving Davis -> San JoseFriday (3/18) at 3:30...	57 m		
9	9	#drive03192022Driving Davis —> San Jose (Safe...	1 h	0	2022-03-19

4.3 Weekday of the Ride

For obtaining the day of week according to the ride date, there is a function called “find_weekday” built. The function “find_weekday” takes a datetime object as input and outputs a string containing the day of the week. The algorithm of this function is, because the datetime object itself has an instant method “.weekday()” that returns a double number from 0 to 6 where 0 means “Monday” and 6 means “Sunday”, if and else is applied for recognize the double number and convert it to the string containing the day of the week.

```
In [62]: def find_weekday(dt):      ##### function for searching for days of the week
weekdays = dt.weekday()
if weekdays == 0:
    answer = "Monday"
elif weekdays == 1:
    answer = "Tuesday"
elif weekdays == 2:
    answer = "Wednesday"
elif weekdays == 3:
    answer = "Thursday"
elif weekdays == 4:
    answer = "Friday"
elif weekdays == 5:
    answer = "Saturday"
else:
    answer = "Sunday"
return(answer)

df2 = pd.DataFrame(index=range(len(df)),columns=range(len(df.columns)))
df2.columns = df.columns

df2['index'] = df['index']
df2['content'] = df['content']
df2['timeofposting'] = df['timeofposting']
df2['rideordrive'] = '' # indicates whether the post is from a driver or rider (driver
df2['dateofride'] = '' # if rider posts more than one hashtag, assume the first hashtag
df2['weekdayofride'] = ''

def hashtag(word): # finds all occurrence of # in string and add empty space in front o
    index = [i for i, w in reversed(list(enumerate(word))) if w == '#'] # List of index
```

```

for i in index:
    word = word[:i] + " " + word[i:]

return word

for i in range(len(df)):
    l = list()

    w = df.iloc[i, 1]

    w = hashtag(w)
    w = re.split(r" ", w)

    for j in w:
        if len(j)>0 and j[0]=='#':
            l.append(j)
        else:
            None

    alphahash = r'^A-Za-z#+'
    numeric = r'^0-9+' # if trying to take dates from all hashtags in a single post,
    rd = re.sub(alphahash, '', str(l))
    date = re.sub(numeric, '', str(l))

    # code to get 'rideordrive'
    if rd[0:5] == "#driv":
        df2['rideordrive'][i] = 0
    elif rd[0:5] == "#ride":
        df2['rideordrive'][i] = 1
    else:
        None

    # code to get all dates from a single post (not needed for now)
    #date = re.split('#', date)
    #date = ["#" + date for date in date] # add # in front of every element in list
    #date = [ele for ele in date if ele != '#'] # remove every element with only #

    # code to get 'dateofride'
    if len(date) > 0:
        if len(date) > 5: # greater than six since "mmddyy" has length six
            if len(date) == 6: # if element has format 'mmddyy'
                date = date[:4] + "20" + date[4:] # e.g. change year 22 to 2022
            if len(date) > 8: # if element has format other than mmddyyyy
                date = date[:8] # set element to format #mmddyyyy

            dte = datetime.strptime(date, '%m%d%Y')
            #####
            weekd = find_weekday(dte) ##### Jianing add for find the day of
            #####

            date = dte.date()
            if date.year > 2012 and date.year < 2025: # filter out invalid dates (e.g.
                df2['dateofride'][i] = date
                df2['weekdayofride'][i] = weekd

        else:
            date = ''

df2.head(10)

```

C:\Users\user\AppData\Local\Temp\ipykernel_2740\1479067934.py:59: SettingWithCopyWarning:
g:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df2['rideordrive'][i] = 0
```

C:\Users\user\AppData\Local\Temp\ipykernel_2740\1479067934.py:84: SettingWithCopyWarning:
g:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df2['dateofride'][i] = date
```

C:\Users\user\AppData\Local\Temp\ipykernel_2740\1479067934.py:85: SettingWithCopyWarning:
g:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df2['weekdayofride'][i] = weekd
```

C:\Users\user\AppData\Local\Temp\ipykernel_2740\1479067934.py:61: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df2['rideordrive'][i] = 1
```

Out[62]:

	index	content	timeofposting	rideordrive	dateofride	weekdayofride
0	0	Not sure if anyone is interested, but I'm driv...	15 March 2020	0	2020-03-17	Tuesday
1	1	#drive03242022Driving Davis -> LA (K-town)Thur...	32 m	0	2022-03-24	Thursday
2	2	Hi!! Looking for a ride from Davis to Riversid...	34 m	1	2022-03-17	Thursday
3	3	#ride03182022 or anytime this week or weekend ...	38 m	1	2022-03-18	Friday
4	4	Looking for a ride from Davis to Santa Barbara...	39 m	1	2022-03-17	Thursday
5	5	#drive03182022Driving Davis->Cupertino/Santa C...	1 d	0	2022-03-18	Friday
6	6	#drive03182022 Davis->East LA Leaving @5:30PM...	48 m	0	2022-03-18	Friday
7	7	Hi! I'm looking for a ride from Davis to UCLA ...	9 h	1	2022-03-18	Friday
8	8	Driving Davis -> San JoseFriday (3/18) at 3:30...	57 m			
9	9	#drive03192022Driving Davis ——> San Jose (Safe...	1 h	0	2022-03-19	Saturday

4.4 Price

The ‘price’ column contains the price offered by the user. In most cases, only drivers will have a value for ‘price’ column, since it’s usual for sellers of a service, in this case drivers, to price the service to customers. Therefore, the process to find this data is only done when the post is written by a driver (‘R/D’ == 1).

The methods used to get ‘price’ data is by first locating the dollar sign (\$). If the type of the character following the dollar sign is numeric, take the dollar sign and the two following characters as the ‘price’ data, since most rides will not cost more than \$100. If the price is two digits, the ‘price’ data would look like ‘\$15’ for example; if the price is one digit, the data would either be a \$ + price + empty space (e.g. ‘ \$8 ’) or \$ + price + random character (e.g. ‘\$8/’). If the next character after the dollar sign is an empty string, the type of character following the empty string is checked; if it is numeric, then the dollar sign and the next three characters are used as ‘price’ data. In this case, if the price is two digits, the data would be in a format \$ + empty space + price, or ‘\$ 15’ for example. If the price is one digit, the data may either include empty space at the end (e.g. ‘ \$8 ’) or a random character (e.g. ‘ \$8?’). When neither of the two following characters after the dollar sign are numeric, the character right before the dollar sign is checked, to consider the unusual possibility of having the format ‘8\$’. If the type of the character is numeric, then the two characters followed by the dollar sign are used as ‘price’ data. Similar to before, the possible format of the ‘price’ data in this case would look like ‘15\$’, ‘ 8\$’, or with a random character in the beginning, ‘:8\$’. If there is no numeric value before or after the dollar sign or if the post does not have the dollar sign in general, the ‘price’ data is kept empty.

In [63]:

```
#finding the price
#Valid

price=list()
```



```
list_posts0=df["content"]
for w in list_posts0:
    tmp=list(w.lower())
    e=0
    while tmp[e]!="$" and e<(len(tmp)-2):
        e=e+1
    if tmp[e]=="$" and e<(len(tmp)-2):
        try:
            int(tmp[e+1]+tmp[e+2])
            price.append(int(tmp[e+1]+tmp[e+2]))
        except:
            try:
                int(tmp[e+1])
                if int(tmp[e+1])>5:
                    price.append(int(tmp[e+1]))
                else:
                    tmppr=int(tmp[e+1])+10
                    print(tmppr)
                    price.appen(tmppr)
            except:
                price.append(0)
    else:
        price.append(0)
df2["price"]=price
df2.head(10)
```

13

Out[63]:

	index	content	timeofposting	rideordrive	dateofride	weekdayofride	price
0	0	Not sure if anyone is interested, but I'm driv...	15 March 2020	0	2020-03-17	Tuesday	35
1	1	#drive03242022Driving Davis -> LA (K-town)Thur...	32 m	0	2022-03-24	Thursday	55
2	2	Hi!! Looking for a ride from Davis to Riversid...	34 m	1	2022-03-17	Thursday	0
3	3	#ride03182022 or anytime this week or weekend ...	38 m	1	2022-03-18	Friday	0
4	4	Looking for a ride from Davis to Santa Barbara...	39 m	1	2022-03-17	Thursday	0
5	5	#drive03182022Driving Davis->Cupertino/Santa C...	1 d	0	2022-03-18	Friday	15
6	6	#drive03182022 Davis->East LA Leaving @5:30PM...	48 m	0	2022-03-18	Friday	45
7	7	Hi! I'm looking for a ride from Davis to UCLA ...	9 h	1	2022-03-18	Friday	0
8	8	Driving Davis -> San JoseFriday (3/18) at 3:30...	57 m				15
9	9	#drive03192022Driving Davis ——> San Jose (Safe...	1 h	0	2022-03-19	Saturday	0

4.5 Location/Destination

Before coding to find the location and destination, we use the “re” package to add spaces in front of every capital letters, numbers and non-word characters, and after the numbers and non-word characters. This allows us to better apply our algorithm to extract the location and destination.

To extract location and destination data from the post content, there is a function called “find_loc_desti” created. The function takes an index value, which indicates the position of methods deciding symbols or words located, and a list object that contains all the information from the post content excluding the spaces.

Through observing the all the posts’ contents, we found that there are some special symbols or words such as five different types of arrows like “-->”, “->”, “→”, a dash “-”, the word “from”, the word “to”, etc., which usually have the destination and starting location of the ride attached to it or within a limited range from it. For example, for all types of arrows, the dash and the word “to”, the

destination is usually attached in the back and the starting location is usually attached in the front. However, the word “from” is a special case. It usually has the starting location attached to it and then, there is a “to” word that has the destination attached to it. As a result, by using the for loop to get contents from each post and turning the contents into a list of words and symbols, if and else is applied to locate the special symbol or word, and then the function “find_loc_desti” is applied to find drivers and riders desired destination and the starting location of the ride.

In addition, when we get the city name of starting location or destination, we include most appeared cases in the “find_loc_desti” function such as “san”, “santa” with a word attached to it or the city abbreviation. By using the if and else, we are able to include the cases and obtain most of the names of the cities.

In [64]:

```
posts = df2["content"]
index = 0

## Add space between all capital letters, and non-characters so we can fix the issue th
## from web scraping, we have some post content have their lines connected without a sp

for i in posts:
    posts[index]=(re.sub(r"(?<=\w)([A-Z]|[0-9]|\W+)", r" \1", i)).lower()
    index = index +1

index = 0
for j in posts:
    posts[index]=(re.sub(r"(?<=\w)([0-9]|\W+)", r"\1 ", j)).lower()
    index = index +1

## Code Reference: https://stackoverflow.com/questions/199059/a-pythonic-way-to-insert-
```

C:\Users\user\AppData\Local\Temp\ipykernel_2740\1646263538.py:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
posts[index]=(re.sub(r"(?<=\w)([A-Z]|[0-9]|\W+)", r" \1", i)).lower()
```

C:\Users\user\AppData\Local\Temp\ipykernel_2740\1646263538.py:13: SettingWithCopyWarnin
g:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
posts[index]=(re.sub(r"(?<=\w)([0-9]|\W+)", r"\1 ", j)).lower()
```

In [28]:

```
def iterative_levenshtein(s, t):
    """
        iterative_levenshtein(s, t) -> ldist
        ldist is the Levenshtein distance between the strings
        s and t.
        For all i and j, dist[i,j] will contain the Levenshtein
        distance between the first i characters of s and the
        first j characters of t
    """

    rows = len(s)+1
    cols = len(t)+1
    dist = [[0 for x in range(cols)] for x in range(rows)]

    # source prefixes can be transformed into empty strings
    # by deletions:
    for i in range(1, rows):
        dist[i][0] = i

    # target prefixes can be created from an empty source string
    # by inserting the characters
    for i in range(1, cols):
        dist[0][i] = i

    for col in range(1, cols):
        for row in range(1, rows):
            if s[row-1] == t[col-1]:
                cost = 0
            else:
```

```

        cost = 1
        dist[row][col] = min(dist[row-1][col] + 1,      # deletion
                             dist[row][col-1] + 1,      # insertion
                             dist[row-1][col-1] + cost) # substitution

    #for r in range(rows):
    #print(dist[r])

    return dist[row][col]

```

In [65]:

```

#filter locations

Cities=["davis","san francisco","san jose","redwood","santa clara", "east bay","south b
"berkeley", "los angeles", "san ramon", "san diego", "orange county", "sacramen
"oakland", "fresno", "long beach","bakersfield","anaheim","stockton","riverside
"santa barbara","santa ana", "san bernardino", "cupertino", "oxnard", "rosevill
"sf","la","sb","sj","sd","marced","san mateo","costco",'milpitas','san leandro'
'ontario','pleasanton','san fernando','santa cruz"]

def f(w,C,P):
    abb=pd.DataFrame()
    abb["abb"]=["sf","sd","sj","la"]
    abb["city"]=["san francisco","san diego", "san jose", "los angeles"]
    c=C[0]
    tmp=iterative_levenshtein(w, c)
    ratio=tmp/len(c)
    L=c
    for c in C[1:]:
        tmp1=iterative_levenshtein(w,c)
        if tmp1<tmp:
            tmp=tmp1
            ratio=tmp1/len(c)
            L=c
        else:
            tmp=tmp
            ratio=ratio
            L=L
    if ratio<=P:
        if L in list(abb["abb"]):
            L=abb.loc[abb["abb"]==L,"city"].unique()[0]
            return L
        else:
            return L
    else:
        return "NaN"

```

In [30]:

```

### Function to find the location and destination given the post content and the index
### Now, the function can find city name starts with "los", "east", "san", "santa", "so
### And, it can also find city abbreviation starts with "l" and "s"

### Attention: this function only applies to the formats that is like "(location) (symbo

def find_loc_desti(index, post_split):
    if post_split[index-2] == "los" or post_split[index-2] == "east" or post_split[inde
        location = (post_split[index-2]+" "+post_split[index-1])
        if post_split[index+1] == "los" or post_split[index+1] == "east" or post_split[
            destination = (post_split[index+1]+" "+post_split[index+2])
        elif post_split[index+1] == "l" or post_split[index+1] == "s":
            destination = (post_split[index+1]+post_split[index+2])
        else:
            destination = post_split[index+1]
    elif post_split[index-2] == "l" or post_split[index-2] == "s":
        location = (post_split[index-2]+post_split[index-1])
        if post_split[index+1] == "los" or post_split[index+1] == "east" or post_split[
            destination = (post_split[index+1]+" "+post_split[index+2])
        elif post_split[index+1] == "l" or post_split[index+1] == "s":
            destination = (post_split[index+1]+post_split[index+2])
        else:
            destination = post_split[index+1]
    else:
        location = post_split[index-1]
        if post_split[index+1] == "los" or post_split[index+1] == "east" or post_split[
            destination = (post_split[index+1]+" "+post_split[index+2])
        elif post_split[index+1] == "l" or post_split[index+1] == "s":

```

```

        destination = (post_split[index+1]+post_split[index+2])
    else:
        destination = post_split[index+1]

    return [location, destination]

```

In [66]:

```

### Getting Destination and Location from drivers' posts

import re

destination = []
location = []
index = 0
post_split = ""
index_loc = 0

### Basically I am playing with indeces to find the names of destinations and locations
### With the function defined above

### If the post content's format is not normal, then the the code is going to return "N"
### The post contents have many formats but mostly are the followings

sep=["->", "-->", "->", ">", "—>", "-->", "from", "to"]
for post in posts:
    post_split = post.split()
    if sep[0] in post_split:
        index = post_split.index(">")
        [loc, desti] = find_loc_desti(index, post_split)
        destination.append(f(loc, Cities, 0.50))
        location.append(f(desti, Cities, 0.50))

    elif sep[1] in post_split:
        index = post_split.index("-->")
        [loc, desti] = find_loc_desti(index, post_split)
        destination.append(f(loc, Cities, 0.50))
        location.append(f(desti, Cities, 0.50))

    elif sep[2] in post_split:
        index = post_split.index("->")
        [loc, desti] = find_loc_desti(index, post_split)
        destination.append(f(loc, Cities, 0.50))
        location.append(f(desti, Cities, 0.50))

    elif sep[3] in post_split:
        index = post_split.index(">")
        [loc, desti] = find_loc_desti(index, post_split)
        destination.append(f(loc, Cities, 0.50))
        location.append(f(desti, Cities, 0.50))

    elif sep[4] in post_split:
        index = post_split.index("—>")
        [loc, desti] = find_loc_desti(index, post_split)
        destination.append(f(loc, Cities, 0.50))
        location.append(f(desti, Cities, 0.50))

    elif sep[5] in post_split:
        index = post_split.index("-->")
        [loc, desti] = find_loc_desti(index, post_split)
        destination.append(f(loc, Cities, 0.50))
        location.append(f(desti, Cities, 0.50))

#### from is a special case because the format usually is "from (location) to (destination)"

    elif sep[6] in post_split:
        index = post_split.index("from")
        if "to" in post_split:
            if post_split[index+1] == "los" or post_split[index+1] == "east" or post_split[index+1] == "west":
                loc=(post_split[index+1]+" "+post_split[index+2])
                location.append(f(loc, Cities, 0.50))
                index_loc = post_split.index("to")
                if post_split[index_loc+1] == "los" or post_split[index_loc+1] == "east" or post_split[index_loc+1] == "west":
                    desti=(post_split[index_loc+1]+" "+post_split[index_loc+2])
                    destination.append(f(desti, Cities, 0.50))
                elif post_split[index_loc+1] == "l" or post_split[index_loc+1] == "s":
                    desti=(post_split[index_loc+1]+post_split[index_loc+2])
                    destination.append(f(desti, Cities, 0.50))
            else:
                desti=post_split[index_loc+1]
                destination.append(f(desti, Cities, 0.50))

```

```
elif post_split[index+1] == "l" or post_split[index+1] == "s":
    loc=(post_split[index+1]+post_split[index+2])
    location.append(f(loc,Cities,0.50))
    index_loc = post_split.index("to")
    if post_split[index_loc+1] == "los" or post_split[index_loc+1] == "east":
        desti=(post_split[index_loc+1]+" "+post_split[index_loc+2])
        destination.append(f(desti,Cities,0.50))
    elif post_split[index_loc+1] == "l" or post_split[index_loc+1] == "s":
        desti=(post_split[index_loc+1]+post_split[index_loc+2])
        destination.append(f(desti,Cities,0.50))
    else:
        desti=post_split[index_loc+1]
        destination.append(f(desti,Cities,0.50))
else:
    loc=post_split[index+1]
    location.append(f(loc,Cities,0.50))
    index_loc = post_split.index("to")
    if post_split[index_loc+1] == "los" or post_split[index_loc+1] == "east":
        desti=(post_split[index_loc+1]+" "+post_split[index_loc+2])
        destination.append(f(desti,Cities,0.50))
    elif post_split[index_loc+1] == "l" or post_split[index_loc+1] == "s":
        desti=(post_split[index_loc+1]+post_split[index_loc+2])
        destination.append(f(desti,Cities,0.50))
    else:
        desti=post_split[index_loc+1]
        destination.append(f(desti,Cities,0.50))
else:
    location.append(None)
    destination.append(None)

### Some people may just drop the "from"

elif sep[7] in post_split:
    index = post_split.index("to")
    [loc,desti] = find_loc_desti(index, post_split)
    destination.append(f(loc,Cities,0.50))
    location.append(f(desti,Cities,0.50))
elif "-" in post_split:
    index = post_split.index("-")
    [loc,desti] = find_loc_desti(index, post_split)
    destination.append(f(loc,Cities,0.50))
    location.append(f(desti,Cities,0.50))
else:
    destination.append("NaN")
    location.append("NaN")
df2["start"] = location
df2["destination"] = destination
df2.head(10)
```

Out[66]:

	index	content	timeofposting	rideordrive	dateofride	weekdayofride	price	start	destination
0	0	not sure if anyone is interested , ...	15 March 2020	0	2020-03-17	Tuesday	35	NaN	NaN
1	1	#drive 0 3 2 4 2 0 2 2 driving davi...	32 m	0	2022-03-24	Thursday	55	los angeles	davis
2	2	hi !! looking for a ride from davis...	34 m	1	2022-03-17	Thursday	0	davis	riverside
3	3	#ride 0 3 1 8 2 0 2 2 or anytime ...	38 m	1	2022-03-18	Friday	0	santa barbara	NaN
4	4	looking for a ride from davis to ...	39 m	1	2022-03-17	Thursday	0	davis	santa barbara

	index	content	timeofposting	rideordrive	dateofride	weekdayofride	price	start	destination
5	5	#drive 0 3 1 8 2 0 2 2 driving davi...	1 d	0	2022-03-18	Friday	15	cupertino	davis
6	6	#drive 0 3 1 8 2 0 2 2 davis -> east...	48 m	0	2022-03-18	Friday	45	east bay	davis
7	7	hi ! i ' m looking for a ride from ...	9 h	1	2022-03-18	Friday	0	davis	NaN
8	8	driving davis -> san jose friday (3 /...	57 m				15	san jose	davis
9	9	#drive 0 3 1 9 2 0 2 2 driving davi...	1 h	0	2022-03-19	Saturday	0	san jose	davis

4.6 Post Date/Time/Weekday

When we obtaining the post date, post imte and post day of week, we use a for loop to get every "time" variable from the "time" column of the dataset and use if and else to distinguish between the three cases: the "time" variable only contains hour value from the scrap time, the "time" only have day value from the scrap time, and the "time" have day, month, hour and minute values. And before the loop runs, we also initialize the scrap time by turning it to a datetime object. Thus, by using the datetime package and the timedelta package, we are able to do datetime object calculation and convert the day value and the hour value into a datetime object. Then, we separate the date and the time from the datetime object and add them to the dataset. The weekday obtaining method is similar to the method in "Weekday of Ride".

In [67]:

```
from datetime import timedelta, datetime

scrap_time = datetime.strptime("14 March 2022 at 22:00", '%d %B %Y at %H:%M') ## Change
post_time00 = []
post_date00 = []
post_weekday = [] ## We need this for calculating the weekdays

for time in time_post:
    time_split = time.split()
    if len(time_split) <= 2:
        if time[-1] == 'm':
            minute = int(time_split[0])
            dt = scrap_time+timedelta(minutes=minute)
            post_time00.append(dt.time().strftime("%H:%M")) ## Get the time in format
            post_date00.append(dt.date()) ## Get the date object
            wd = find_weekday(dt)
            post_weekday.append(wd)

        elif time[-1] == 'h': ## converting hours
            hour = int(time_split[0])
            dt = scrap_time+timedelta(hours=hour)
            post_time00.append(dt.time().strftime("%H:%M")) ## Get the time in format
            post_date00.append(dt.date()) ## Get the date object
            wd = find_weekday(dt)
            post_weekday.append(wd)

        elif time[-1] == 'd': ## converting days
            day = int(time_split[0])
            dt = scrap_time+timedelta(days=day)
            post_time00.append(dt.time().strftime("%H:%M"))
            post_date00.append(dt.date())
            wd = find_weekday(dt)
```

```

        post_weekday.append(wd)
    else:
        post_weekday.append(None)
        post_time00.append(None)
        post_date00.append(None)

    elif len(time_split) == 4:    ## Converting normal date and time into datetime objec
        if ("December" not in time) & ("November" not in time) & ("October" not in time
            time = "2022 "+time
        else:
            time = "2021 "+time
            dt = datetime.strptime(time,'%Y %d %B at %H:%M')
            post_date00.append(dt.date())
            post_time00.append(dt.time().strftime("%H:%M"))
            wd = find_weekday(dt)
            post_weekday.append(wd)

    else:
        post_weekday.append(None)
        post_time00.append(None)
        post_date00.append(None)

#### Code Reference 1: https://strftime.org/
#### Code Reference 2: https://www.pressthered.com/adding_dates_and_times_in_python/

df2["dateofpost"] = post_date00
df2["timeofpost"] = post_time00
df2["weekdayofpost"] = post_weekday
df2.head(10)
```

Out[67]:

	index	content	timeofposting	rideordrive	dateofride	weekdayofride	price	start	destination
0	0	not sure if anyone is interested , ...	15 March 2020	0	2020-03-17	Tuesday	35	NaN	NaN
1	1	#drive 0 3 2 4 2 0 2 2 driving davi...	32 m	0	2022-03-24	Thursday	55	los angeles	davis
2	2	hi !! looking for a ride from davis...	34 m	1	2022-03-17	Thursday	0	davis	riverside
3	3	#ride 0 3 1 8 2 0 2 2 or anytime ...	38 m	1	2022-03-18	Friday	0	santa barbara	NaN
4	4	looking for a ride from davis to ...	39 m	1	2022-03-17	Thursday	0	davis	santa barbara
5	5	#drive 0 3 1 8 2 0 2 2 driving davi...	1 d	0	2022-03-18	Friday	15	cupertino	davis
6	6	#drive 0 3 1 8 2 0 2 2 davis -> east...	48 m	0	2022-03-18	Friday	45	east bay	davis
7	7	hi ! i ' m looking for a ride from ...	9 h	1	2022-03-18	Friday	0	davis	NaN
8	8	driving davis -> san jose	57 m				15	san jose	davis

	index	content	timeofposting	rideordrive	dateofride	weekdayofride	price	start	destination
		friday (3 /...							
9	9	#drive 0 3 1 9 2 0 2 2 driving davi...	1 h	0	2022-03-19	Saturday	0	san jose	davis

In [68]:

```
df100=df2
```

In [69]:

```
df100=df100.dropna()
df100= df100.drop(["content","timeofposting"],axis=1)
df100.head(10)
```

Out[69]:

	index	rideordrive	dateofride	weekdayofride	price	start	destination	dateofpost	timeofpost
1	1	0	2022-03-24	Thursday	55	los angeles	davis	2022-03-14	22:32
2	2	1	2022-03-17	Thursday	0	davis	riverside	2022-03-14	22:34
3	3	1	2022-03-18	Friday	0	santa barbara	NaN	2022-03-14	22:38
4	4	1	2022-03-17	Thursday	0	davis	santa barbara	2022-03-14	22:39
5	5	0	2022-03-18	Friday	15	cupertino	davis	2022-03-15	22:00
6	6	0	2022-03-18	Friday	45	east bay	davis	2022-03-14	22:48
7	7	1	2022-03-18	Friday	0	davis	NaN	2022-03-15	07:00
8	8				15	san jose	davis	2022-03-14	22:57
9	9	0	2022-03-19	Saturday	0	san jose	davis	2022-03-14	23:00
10	10	0	2022-03-18	Friday	14	san jose	NaN	2022-03-18	22:00

◀

▶

5 Data Analysis

The data can be visualised by various methods.

In [70]:

```
### Top 10 rows of the code as a table

import matplotlib.pyplot as plt
import six

df20 = df100.head(10)

### The function set the table's background color for each row and the title
def render_mpl_table(data, col_width=3.0, row_height=0.625, font_size=14,
                    header_color='#40466e', row_colors=['#f1f1f2', 'w'], edge_color='w',
                    bbox=[0, 0, 1, 1], header_columns=0,
                    ax=None, **kwargs):
    if ax is None:
        size = (np.array(data.shape[:-1]) + np.array([0, 1])) * np.array([col_width, row_height])
        fig, ax = plt.subplots(figsize=size)
        ax.axis('off')

    mpl_table = ax.table(cellText=data.values, bbox=bbox, colLabels=data.columns, **kwargs)

    mpl_table.auto_set_font_size(False)
```

```
mpl_table.set_fontsize(font_size)

for k, cell in six.iteritems(mpl_table._cells):
    cell.set_edgecolor(edge_color)
    if k[0] == 0 or k[1] < header_columns:
        cell.set_text_props(weight='bold', color='w')
        cell.set_facecolor(header_color)
    else:
        cell.set_facecolor(row_colors[k[0]%len(row_colors) ])
return ax

render_mpl_table(df20, header_columns=0, col_width=3.0)
```

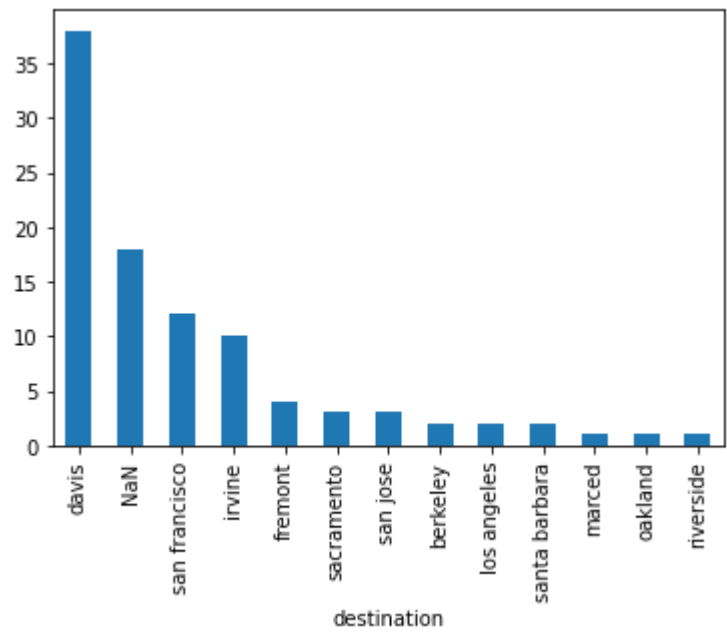
Out[70]: <AxesSubplot:>

index	rideordrive	dateofride	weekdayofride	price	start	destination	dateofpost	timeofpost	weekdayofpost
1	0	2022-03-24	Thursday	55	los angeles	davis	2022-03-14	22:32	Monday
2	1	2022-03-17	Thursday	0	davis	riverside	2022-03-14	22:34	Monday
3	1	2022-03-18	Friday	0	santa barbara	NaN	2022-03-14	22:38	Monday
4	1	2022-03-17	Thursday	0	davis	santa barbara	2022-03-14	22:39	Monday
5	0	2022-03-18	Friday	15	cupertino	davis	2022-03-15	22:00	Tuesday
6	0	2022-03-18	Friday	45	east bay	davis	2022-03-14	22:48	Monday
7	1	2022-03-18	Friday	0	davis	NaN	2022-03-15	07:00	Tuesday
8				15	san jose	davis	2022-03-14	22:57	Monday
9	0	2022-03-19	Saturday	0	san jose	davis	2022-03-14	23:00	Monday
10	0	2022-03-18	Friday	14	san jose	NaN	2022-03-18	22:00	Friday

From the bar plot of “destination”, we find that riders prefer to have their destination set on San Francisco, Irvine, and Fremont besides Davis. And, the drivers like to set their destination on San Francisco, Irvine, and Fremont too besides Davis.

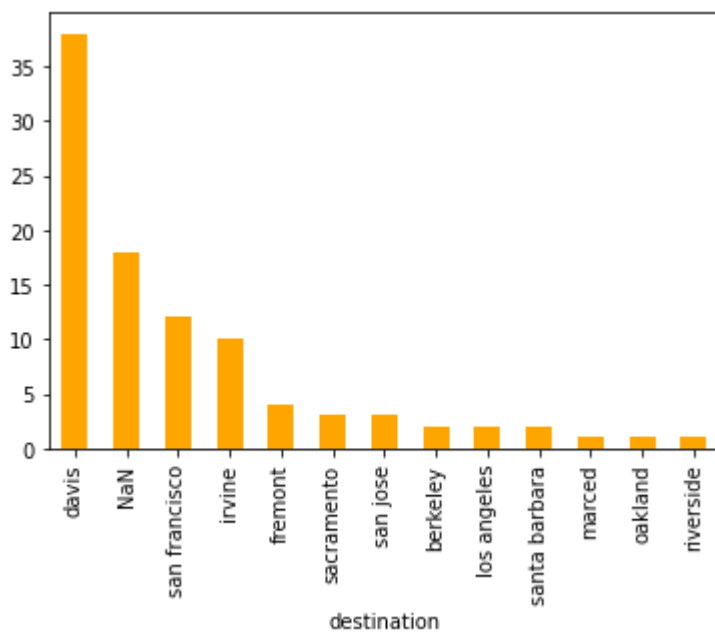
```
In [71]: df3 = (df100.loc[df2["rideordrive"]==1])
df3.value_counts("start").plot.bar() ## Favorability of the starting location for driv
df3.value_counts("destination").plot.bar() ## Favorability of the destination for driv
```

Out[71]: <AxesSubplot:xlabel='destination'>



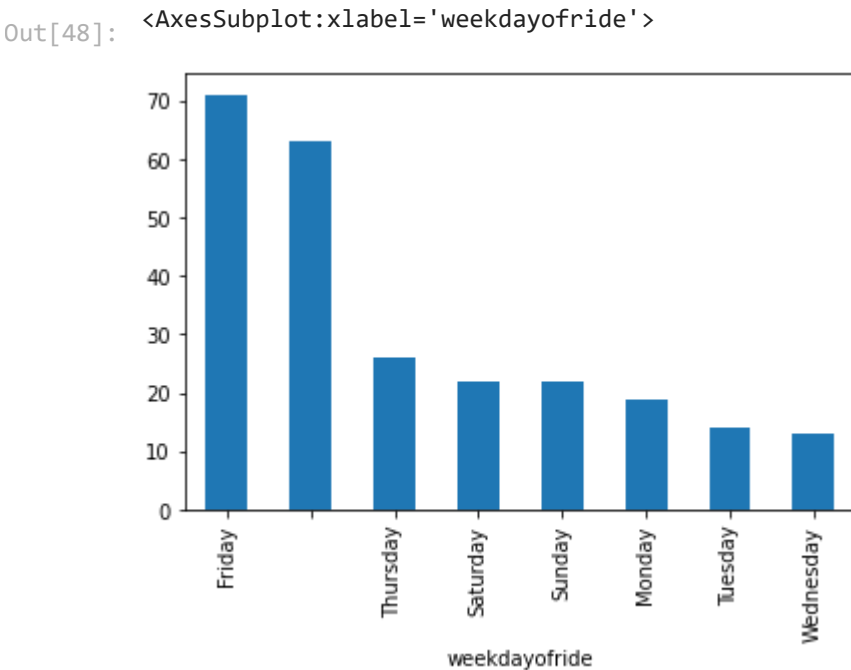
```
In [73]: df4 = (df100.loc[df2["rideordrive"]==0])
df3.value_counts("start").plot.bar(color = "orange") ## Favorability of the destination for drivers
df3.value_counts("destination").plot.bar(color = "orange") ## Favorability of the destination for riders
```

Out[73]: <AxesSubplot:xlabel='destination'>



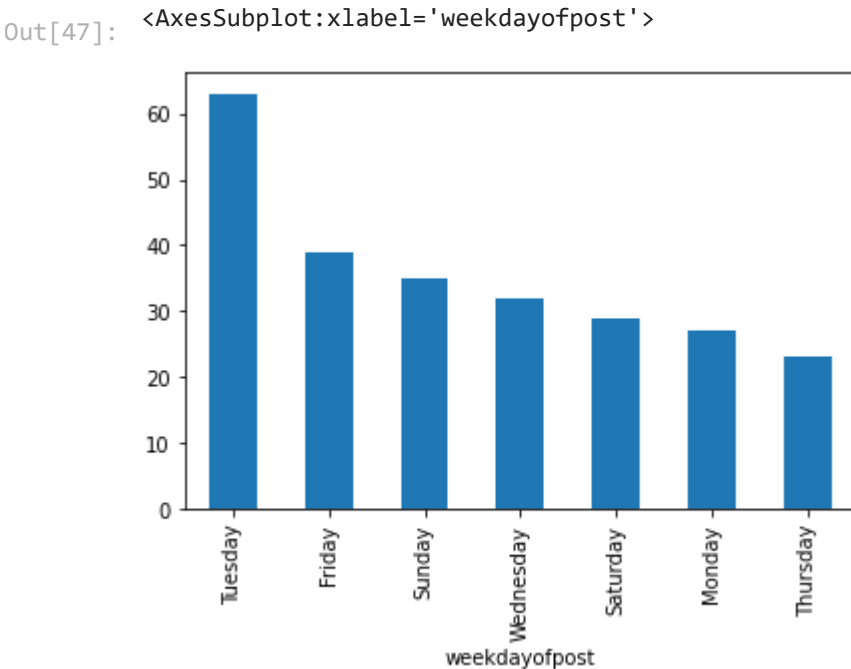
By observing the frequency graph of “weekdayofride”, we find that most people like to have the rides on Friday.

```
In [48]: df2.value_counts("weekdayofride").plot.bar() ### bar plots for the frequency of reques
```



By observing the frequency graph of “weekdayofpost”, we find that most people in the group like to post their needs on Tuesdays.

```
In [47]: df2.value_counts("weekdayofpost").plot.bar() ### bar plots for the frequency of post i
```



6 Conclusion

During this project, there were countless difficulties encountered during the web scraping and data cleaning. One of the biggest challenges existed in the process of web scraping posts and other relevant information from Facebook, since Facebook's website is built to minimize the possibility of web scraping. After successfully gathering the data from Facebook, the data transformation was the second hurdle of the project. Each type of data required for data analysis demanded different ways of transformation, resulting in considering and developing codes to find relevant information from the posts. With the data gathered from these steps, the final data analysis was able to successfully understand the users' behaviors and develop more models to predict future actions of the users. From these series of actions, another step to the goal of making a website with higher efficiency and accessibility for group members was able to be achieved.

7 Code Reference

- <https://www.facebook.com/groups/UCDavisRideSharing>
- https://github.com/MariyaSha/WebscrapingFacebook/blob/main/FB_Scraper.ipynb
- <https://stackoverflow.com>
- <https://strftime.org/>
- https://www.pressthered.com/adding_dates_and_times_in_python/
- <https://docs.python.org/3/library/datetime.html#datetime.time.minute>
- <https://stackoverflow.com/questions/199059/a-pythonic-way-to-insert-a-space-before-capital-letters>